

Cryptography and Network Security

Integrated Situation — Technical Report

ETTCS801 | ULK Polytechnic Institute

Seba Michel

Registration Number: 4202570055

September 29, 2026

Contents

1 Introduction.....	2
2 Risk Assessment	2
2.1 Assets, vulnerabilities and consequences.....	2
2.2 Risk ranking	2
2.3 Recommended controls.....	2
3 Encryption and Integrity Application.....	3
3.1 Design	3
3.2 Test evidence.....	3
4 Network Traffic Filtering.....	3
4.1 Laboratory setup.....	3
4.2 Firewall rules.....	3
4.3 Test procedure and results.....	4
5 Conclusion	4

1 Introduction

A polytechnic institute stores student records on a central server and transfers files between two campuses. A security review found weak staff passwords, outdated software, unencrypted file transfers and guest-network access to the records server, alongside repeated connection attempts from an unfamiliar external address. This report documents the risk assessment, the Python encryption and integrity toolkit, and the firewall rules produced to address these findings. All source files are available in the GitHub repository (Section 5).

2 Risk Assessment

2.1 Assets, vulnerabilities and consequences

Asset	Vulnerability	Possible consequence
R1 Staff accounts	Weak passwords; repeated external login attempts	Account takeover; records read, altered or deleted; misuse under a staff identity.
R2 Records server	Guest network can reach server; outdated software	Exploitation of flaws; data breach, ransomware, service outage; legal and reputational harm.
R3 Files in transit	Unencrypted file transfer	Eavesdropping or man-in-the-middle; records disclosed or silently modified.

2.2 Risk ranking

Likelihood and impact are scored from 1 (low) to 5 (high); the score is their product.

Rank	Risk	Likelihood	Impact	Score
1	R1 – Weak staff passwords	5	5	25
2	R2 – Guest access / outdated software	4	5	20
3	R3 – Unencrypted transfers	3	4	12

Reasoning. R1 ranks first because attacks are already occurring and weak passwords need little skill to defeat. R2 is second because guest users are untrusted and known exploits exist for outdated software. R3 ranks third because interception requires a network position between campuses.

2.3 Recommended controls

Risk	Control
R1	Strong password policy (min 12 chars), multi-factor authentication, and account lockout.

R2	Network segmentation with firewall rules denying guest access, plus regular patching.
R3	Encrypt files before transfer using AES-256-GCM and SHA-256 integrity checks.

3 Encryption and Integrity Application

3.1 Design

The program `src/secure_records.py` uses Python and the cryptography library.

- Encryption – AES-256-GCM authenticated encryption using a 256-bit key stored securely outside the repository (`~/.ulk_keys/records.key`).
- Decryption – Validates nonces and authentication tags, reporting clean errors on tampering.
- Integrity – Uses SHA-256 baseline hashes stored in `hashes.json` to report *UNCHANGED* or *MODIFIED*.
- Error Handling – Missing files, empty inputs, and corrupt keys are caught gracefully with clean messages and proper exit codes.

3.2 Test evidence

All ten automated unit tests and CLI checks pass successfully:

```
$ python3 -m unittest discover -s tests -v
...
Ran 10 tests in 0.008s
OK
```

4 Network Traffic Filtering

4.1 Laboratory setup

The lab environment uses records server IP `192.168.100.10`, staff network `192.168.10.0/24`, guest network `10.0.99.0/24`, firewall tool `iptables`, and the tested service port `tcp/443`.

4.2 Firewall rules

Rules are managed via a dedicated `RECORDS_FILTER` chain:

1. Accept established and related traffic.
2. Block guest network access to the server.
3. Permit staff network access to the designated service port.
4. Block all other inbound attempts to that service.

```

iptables -A RECORDS_FILTER -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
iptables -A RECORDS_FILTER -s $GUEST_NET -j DROP iptables -A RECORDS_FILTER -s
\($STAFF_NET -p tcp --dport\)$SERVICE_PORT \
    -m conntrack --ctstate NEW -j ACCEPT
iptables -A RECORDS_FILTER -p tcp --dport $SERVICE_PORT -j DROP
iptables -A RECORDS_FILTER -j RETURN iptables -I INPUT 1 -j
RECORDS_FILTER

```

4.3 Test procedure and results

Tested using netcat (nc -zv -w 5):

Test	Source	Expected	Actual
T1 permitted	Staff client	Connection succeeds	Connection to port 443 192.168.100.10 succeeded!
T2 blocked	Guest client	Timeout / dropped	nc: connect to 192.168.100.10 port timed out 443
T3 blocked	Other client	Timeout / dropped	nc: connect to 192.168.100.10 port timed out 443

5 Conclusion

The assessment successfully identified risks and implemented robust solutions through a Python cryptographic toolkit and strict iptables firewall configurations. All tests behaved as expected, validating the security controls.

GitHub repository: <https://github.com/seba-michel/cryptography-network-security-exam>